

Homework 1

Due: 11:59pm on March 20, 2025 (Thursday)

Objectives:

- Review core concepts and limitations of arrays
- Reinforce the notion of Object-Oriented Programming

In this homework, you are to implement a Java class:

```
public class MyArray
```

You are NOT allowed to use any of the Java Collections framework. If you have any imports going on in your class, you will lose points on this assignment. Instead, you must use a `String[]` array as its underlying data structure to store and manipulate words in your class. As we discussed in class, a typical array should never allow any holes. Thus, at any point, this underlying array should never have any holes (e.g., nulls in the middle of indices with strings, etc.). Like Java's `ArrayList` class, 0 can be passed as its initial capacity; this capacity must be used as the value of the length of the underlying array that is being instantiated in constructors. For the no-arg constructor, the initial length of the underlying array should be 10.

You must provide the following constructors and methods in your `MyArray` class. Your method signatures must match what is listed below. Pay attention to the return types and parameter types of each method.

```
public MyArray()
public MyArray(int initialCapacity)
public void add(String text)
public boolean search(String key)
public int size()
public int getCapacity()
public void display()
public void removeDups()
```

Step-by-step guide:

- Think carefully about what methods you need to implement. Additional information about all the necessary methods that you are to implement can be found in `HW1Driver.java` file.
 - When inserting new words, duplicates are allowed.
 - The `add` method should take care of validating words. The definition of word is defined in the next page.
 - If `null` is passed for `add` and `search` methods, do not throw exceptions.
 - The underlying `String[]` array should double its length when necessary.

- The only time to remove duplicates in the array is when `removeDups` method is invoked. As the name of the method indicates, when `removeDups` method is invoked, all of the duplicates should be removed from the underlying `String[]` array. We do not want to waste any memory here.
- The `size` method's worst-case running time complexity must be $O(1)$.
- You are NOT allowed to use the Java Collections Framework.
 - The `Arrays` class is a part of the Java Collections Framework. That means you cannot use the `Arrays` class in your code. Once again, there should be no imports in your code.
- You are NOT allowed to use any other data structures than one `String[]` array.
 - This also means you are not allowed to use two or more `String[]` arrays. Especially, think carefully about how you can implement your `removeDups` method without having any temporary or auxiliary array(s). You should not rely on other kind of Java classes either. In other words, you should find way(s) to manipulate the existing array without using any other data structures and any other Java classes.
 - When increasing array capacity, you certainly do need to create another array. But once you done increasing, then there should be only one array.
- You are NOT allowed to use any sorting algorithms.
- You are more than welcome to use private helper methods. In fact, we encourage you to do so.
- Your clients may choose to convert words in a file to lowercase before inserting them. Thus, you should not convert strings to lowercases in your implementation.
- Implement the class `MyArray` on an IDEs of your choice.
 - In your JavaDoc comments, clearly write worst-case running time complexity using Big-O notation for each of the public methods, including constructors.
 - Submit the source file on Autolab.
 - Follow the Java Coding Conventions as described in <https://www.oracle.com/java/technologies/javase/codeconventions-contents.html>. Note that Autolab will report any violations of the conventions. Make sure you use 4 spaces (and not tabs) for your indentations.
- Test your code extensively!
 - When you run `HW1Driver` with your `MyArray` implementation and the given text file, `prideprejudice.txt`, you should see the output that is in `HW1Driver.java` file. The `HW1Driver.java` file contains very simple test code for you. It is strongly recommended that you write your own test code using another driver (.java file). When running the `HW1Driver` with the provided text file, some IDEs may require you to put the text file into the project (root) folder, not in the `src` (sub-)folder.
 - For `display` method, your code should print all the words in one line and put a space between words.
 - A word is a sequence of letters `[a..zA..Z]`, does not include digits `[0..9]`, and does not include other characters than the sequence of letters `[a..zA..Z]`. For each `String` being added to your array, if it is not a word according to the provided definition, gently ignore the word. Do not throw an exception. Here are some examples of non-words: `abc123`, `a12bc`, `1234`, `ab_c`, `abc_`.

Deliverables:

- Your source code file. Make sure to include your Andrew ID and Name in the JavaDoc comment using author tag. Make sure to not use any package statements, since they cause issues with Autolab. Submit your source code file (MyArray.java) using Autolab (<https://autolab.andrew.cmu.edu>).
- Make sure to remove any print statements that you put for your own debugging.
- The result output generated by Autolab may not have a lot of details and that is intentional. They are intended to be enough for you to narrow down your debugging process to fix any issue(s). Hence, we encourage you to spend some time to think about edge cases you might have missed before resubmitting your code to Autolab.

Grading:

Autolab will grade your assignment as follows.

- Working code: 90 points
- Coding conventions: 10 points
 - We will deduct one point for each coding convention issue detected.

In case you do not pass test case(s), please spend some time to think about edge cases you may have missed and test thoroughly before asking questions.

Autolab will check your coding conventions quite strictly. Make sure to read the Java Coding Conventions document when in doubt.

Autolab will show you the results of its grading within approximately a few minutes of your submission. You may submit multiple times to correct any problems with your assignment. Autolab uses the last submission as your grade.

The Autolab score is not final. The TAs will look at your source code for correctness and design, and deduct points if applicable. The most important criterion is always correctness. It is also critical that your code is efficient and follows the specification properly. Additionally, it should be readable and well organized. This includes proper use of clear comments. Points will be deducted for poor design decisions and unreadable code, etc.

As mentioned in the syllabus, **late submissions will NOT be accepted**. If you have multiple versions of your code file, make sure you do submit the correct version. Make sure to submit a .java file, not a .class file! Do not zip your file.